CONTAINERIZATION AND ORCHESTRATION REPORT

KUBERNETES TASK 3

Student Name: PRINCE RAJ

1. OBJECTIVE

The objective of this practical was to demonstrate container orchestration using Kubernetes by deploying an NGINX application with Deployment and Service manifests, verifying the running Pod, scaling replicas, performing a rolling update, and demonstrating rollback.

2. ENVIRONMENT AND TECHNOLOGIES

Technology / Component	Purpose
Kubernetes	Container orchestration
Docker Desktop Kubernetes	Kubernetes cluster used for the practical
kubectl	Command-line interface for Kubernetes
NGINX	Web application/container used for deployment
YAML manifests	Definition of the Deployment and Service

3. SYSTEM ARCHITECTURE

The application was deployed using a Kubernetes Deployment and exposed through a Kubernetes Service. The Deployment maintained the NGINX Pods, while the Service provided a stable way to reach the application. Port forwarding was used to access the Service from the Windows browser.

Architecture flow:
Browser → localhost:8081 → Port Forward → Kubernetes Service (`nginx-demo-service`) → NGINX Pod(s) → NGINX container

4. DEPLOYMENT MANIFEST

The Deployment manifest defined an application named `nginx-demo`, used the NGINX container image, exposed container port 80, and initially configured one replica. Kubernetes created the corresponding Pod and maintained its state.

Deployment Verification

```
C:\Users\HP\kubernetes-task-1>kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
nginx-demo-6f5d9c55b8-74wjz   1/1     Running   0          2m3s
```

Figure 1. Deployment and running NGINX Pod.

5. SERVICE AND APPLICATION ACCESS

A NodePort Service named `nginx-demo-service` was created. The Service selected Pods using the `app: nginx-demo` label and forwarded traffic to port 80 of the NGINX container. The endpoint output confirmed that the Service was connected to the running Pod.

Service and Endpoint Verification

Service and Endpoints

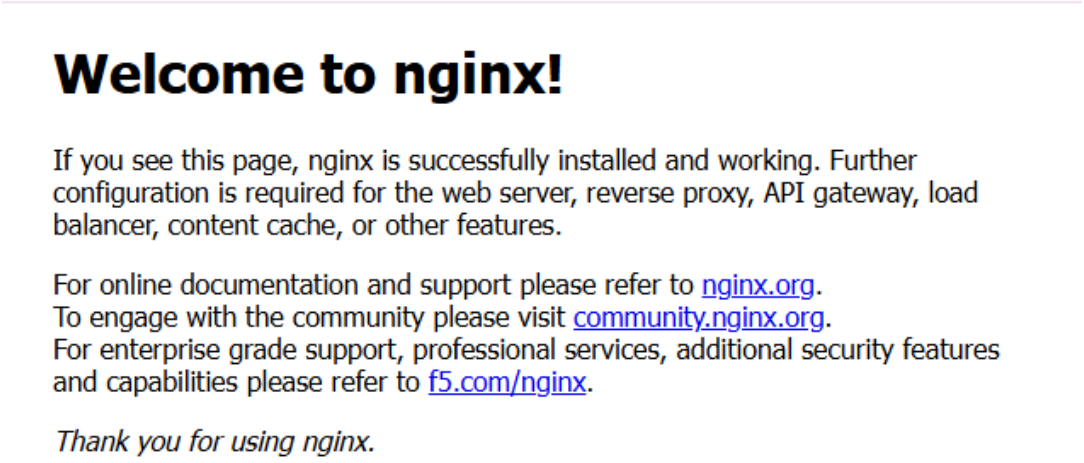
```
C:\Users\HP\kubernetes-task-1>kubectl get services
NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes           ClusterIP   10.96.0.1      <none>        443/TCP        40m
nginx-demo-service   NodePort    10.96.168.87   <none>        80:31450/TCP   10s

C:\Users\HP\kubernetes-task-1>kubectl get endpoints nginx-demo-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                 ENDPOINTS       AGE
nginx-demo-service   10.244.0.5:80   21s
```

Port Forwarding

```
C:\Users\HP\kubernetes-task-1>kubectl port-forward service/nginx-demo-service 8081:80
Forwarding from 127.0.0.1:8081 -> 80
Forwarding from [::1]:8081 -> 80
```

NGINX Application Verification



6. DEPLOYMENT STRATEGY

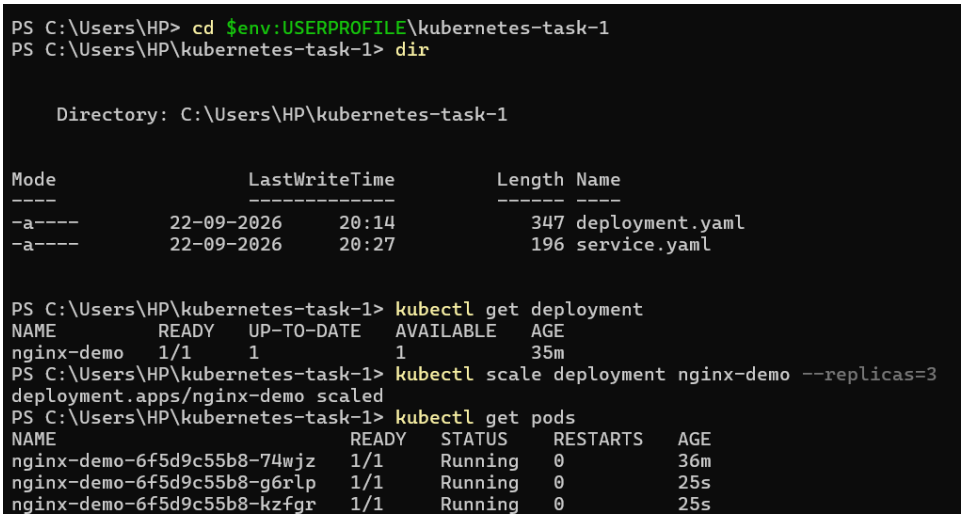
The Deployment was first run with one replica. It was then scaled to three replicas. A rolling update was performed by changing the NGINX image from `nginx:latest` to `nginx:alpine`. Kubernetes replaced the Pods and reported that the Deployment was successfully rolled out. A rollback was then performed, returning the Deployment to the previous image configuration.

7. SCALING RESULTS

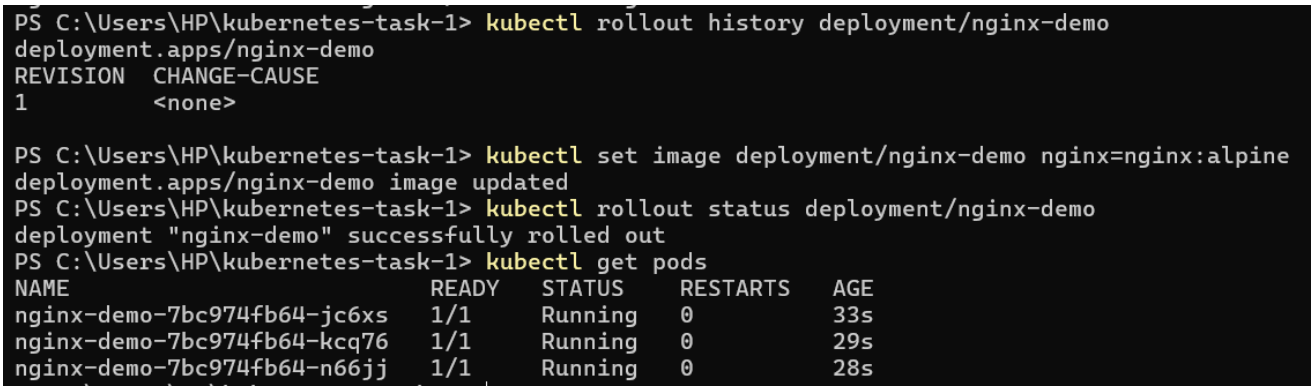
The application was scaled from one replica to three replicas using `kubectl scale deployment nginx-demo --replicas=3`. The verification output showed three Pods with READY status 1/1 and STATUS Running.

Scaling Evidence

Scaling to 3 Replicas



Rolling Update



8. ROLLBACK RESULTS

After the rolling update, the Deployment was rolled back using `kubectl rollout undo deployment/nginx-demo`. The rollback completed successfully. The final verification showed three Running Pods and the Deployment image returned to `nginx:latest`.

Rollback Evidence

Rollback

```
PS C:\Users\HP\kubernetes-task-1> kubectl rollout undo deployment/nginx-demo
Warning: resource deployments/nginx-demo was previously managed with 'kubectl apply'. Rolling back will not update the k
ubectl.kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply
' operations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/nginx-demo rolled back
```

Rollback Verification

```
PS C:\Users\HP\kubernetes-task-1> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
nginx-demo-6f5d9c55b8-c7f8c   1/1     Running   0          3m18s
nginx-demo-6f5d9c55b8-pp2nc   1/1     Running   0          3m25s
nginx-demo-6f5d9c55b8-xck4r   1/1     Running   0          3m21s
PS C:\Users\HP\kubernetes-task-1> kubectl get deployment nginx-demo -o jsonpath='{.spec.template.spec.containers[0].imag
e}'
nginx:latest
```

9. OBSERVATIONS

- The Kubernetes control-plane node was in Ready state before deployment.
- The Deployment successfully created and maintained the NGINX Pod.
- The Service endpoint confirmed connectivity between the Service and Pod.
- The application was accessible from the Windows browser through port forwarding.
- Scaling increased the Deployment from one to three running replicas.
- The rolling update completed successfully and produced a new set of Pods.
- Rollback successfully returned the Deployment to the previous image configuration.
- During browser access, local port 8080 was unavailable, so port 8081 was used successfully.

10. CONCLUSION

The NGINX application was successfully deployed and managed using Kubernetes. The practical demonstrated Deployment and Service manifests, Pod verification, replica scaling, rolling update, rollback, Service endpoint verification, and browser-based application access. The final Kubernetes state maintained three running replicas with the original NGINX image after rollback.